Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

COURSE CODE: DJS23ILPC501 **DATE: 01/10/2026**

COURSE NAME: Computer Networks Laboratory **CLASS: T. Y. B.Tech**

BATCH: I3-1

Experiment No. 8

CO/LO: Implement Transport Layer Protocols.

AIM / OBJECTIVE: To implement sockets using TCP

CLIENT 1 CODE:

```
import socket
import json

# ---------------------------------
# Server IP Address
# ---------------------------------

SERVER_IP = " 10.79.163.94"

SERVER_PORT = 8000

# ---------------------------------
# Create TCP Socket
# ---------------------------------

client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# ---------------------------------
# Connect to Server
# ---------------------------------

client_socket.connect(
    (SERVER_IP, SERVER_PORT)
)

print("\n")
print("=" * 50)
```

```
print("                CLIENT 1")
print("=" * 50)

print("Connected to Server.")

# ---------------------------------
# Enter Array
# ---------------------------------

array_input = input(
    "\nEnter array elements separated by spaces: "
)

# Convert input into integer list
array = list(
    map(
        int,
        array_input.split()
    )
)

print("\nArray entered:", array)

# ---------------------------------
# Send Array to Server
# ---------------------------------

client_socket.sendall(
    json.dumps(array).encode()
)

print("Array sent to Server.")

# ================================================
# RECEIVE CASE 1 RESULT
# ================================================

data = client_socket.recv(4096).decode()

response = json.loads(data)
```

```
print("\n")
print("=" * 50)
print("CASE 1: INDIVIDUAL SORTING")
print("=" * 50)

print(
    "Sorted Array:",
    response["individual_sorted"]
)

# ==================================================
# RECEIVE CASE 2 RESULT
# ==================================================

data = client_socket.recv(4096).decode()

response = json.loads(data)

print("\n")
print("=" * 50)
print("CASE 2: MERGE AND SORT")
print("=" * 50)

print(
    "Final Merged Sorted Array:",
    response["merged_sorted"]
)

# ----------------------------------
# Close connection
# ----------------------------------

client_socket.close()

print("\nConnection closed.")
```

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

CLIENT 1 OUTPUT:

```
==================================================
                  CLIENT 1
==================================================
Connected to Server.

Enter array elements separated by spaces: 8 4 1 5 10

Array entered: [8, 4, 1, 5, 10]
Array sent to Server.


==================================================
CASE 1: INDIVIDUAL SORTING
==================================================
Sorted Array: [1, 4, 5, 8, 10]


==================================================
CASE 2: MERGE AND SORT
==================================================
Final Merged Sorted Array: [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]

Connection closed.
```

CLIENT 2 CODE:

```
import socket
import json

# ---------------------------------
# Server IP Address
# ---------------------------------

SERVER_IP = " 10.79.163.94"

SERVER_PORT = 8000

# ---------------------------------
# Create TCP Socket
# ---------------------------------

client_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# ---------------------------------
# Connect to Server
# ---------------------------------

client_socket.connect(
    (SERVER_IP, SERVER_PORT)
)

print("\n")
print("=" * 50)
print("                CLIENT 2")
print("=" * 50)

print("Connected to Server.")

# ---------------------------------
# Enter Array
# ---------------------------------
```

```
array_input = input(
    "\nEnter array elements separated by spaces: "
)



# Convert input into integer list
array = list(
    map(
        int,
        array_input.split()
    )
)

print("\nArray entered:", array)

# ---------------------------------.
# Send Array to Server
# ---------------------------------.

client_socket.sendall(
    json.dumps(array).encode()
)

print("Array sent to Server.")

# ================================================
# RECEIVE CASE 1 RESULT
# ================================================

data = client_socket.recv(4096).decode(

response = json.loads(data)

print("\n")
print("=" * 50)
print("CASE 1: INDIVIDUAL SORTING")
print("=" * 50)

print(
    "Sorted Array:",
```

```
    response["individual_sorted"]
)

# ==================================================
# RECEIVE CASE 2 RESULT
# ==================================================

data = client_socket.recv(4096).decode()

response = json.loads(data)

print("\n")
print("=" * 50)
print("CASE 2: MERGE AND SORT")
print("=" * 50)

print(
    "Final Merged Sorted Array:",
    response["merged_sorted"]
)

# ---------------------------------
# Close connection
# ---------------------------------

client_socket.close()

print("\nConnection closed.")
```

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

CLIENT 2 OUTPUT:

```
==================================================
                  CLIENT 2
==================================================
Connected to Server.

Enter array elements separated by spaces: 6 1 3 9 11

Array entered: [6, 1, 3, 9, 11]
Array sent to Server.


==================================================
CASE 1: INDIVIDUAL SORTING
==================================================
Sorted Array: [1, 3, 6, 9, 11]


==================================================
CASE 2: MERGE AND SORT
==================================================
Final Merged Sorted Array: [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]

Connection closed.
```

SERVER CODE:

```
import socket

import threading

import json

# ---------------------------------.

# Global dictionary to store arrays from clients

# ---------------------------------.

client_arrays = {}


# Lock used for accessing shared data

lock = threading.Lock()

# ---------------------------------.

# Selection Sort with complete pass-by-pass display

# ---------------------------------.

def selection_sort(arr, name):

    print("\n")

    print("=" * 60)

    print("SELECTION SORT:", name)

    print("=" * 60)

    print("Original Array:", arr)

    n = len(arr)
```

```
# Selection Sort passes

for i in range(n - 1):

    min_index = i

    print("\n--------------------------")

    print("PASS", i + 1)

    print("------------------------------------------")

    print("Starting Array:", arr)

    print("Assume minimum =", arr[min_index])

    print("Minimum index =", min_index)


    # Compare remaining elements

    for j in range(i + 1, n):

        print(

            "Compare",

            arr[j],

            "with",

            arr[min_index]

        )

        if arr[j] < arr[min_index]:

            min_index = j

            print(
```

```
            "New minimum found:",
            arr[min_index],
            "at index",
            min_index
        )
    # Swap minimum element with current element
    if min_index != i:
        print(
            "Swap",
            arr[i],
            "and",
            arr[min_index]
        )
        arr[i], arr[min_index] = (
            arr[min_index],
            arr[i]
        )
    else:
        print("No swap required.")
    print("Array after Pass", i + 1, ":", arr)
print("\nFinal Sorted Array:", arr)
```

```
    print("C" * 60)

    return arr

# ---------------------------------.

# Function to handle each client

# ---------------------------------.

def handle_client(client_socket, address, client_number):

    print("\n")

    print("Client", client_number, "connected.")

    print("Client Address:", address)

    # ------------------------------

    # Receive array from client

    # ------------------------------

    data = client_socket.recv(4096).decode()

    if not data:

        client_socket.close()

        return

    # Convert JSON data into Python list

    array = json.loads(data)

    print("\nArray received from Client", client_number)

    print("Array:", array)

    # Store array
```

```
with lock:
    client_arrays[client_number] = array
print("\nClient", client_number, "array stored.")
print("Waiting for the other client...")
# ------------------------------
# Wait until both clients send arrays
# ------------------------------
while True:

    with lock:
        if len(client_arrays) == 2:
            break
# ================================================
# CASE 1
# INDIVIDUAL SELECTION SORT
# ================================================
print("\n\n")
print("#" * 70)
print("CASE 1: INDIVIDUAL SORTING")
print("#" * 70)
# Make a copy so original array remains unchanged
```

```
individual_array = array.copy()

# Selection Sort

individual_sorted = selection_sort(

    individual_array,

    "Client " + str(client_number)

)

# Send individual sorted array

response = {

    "case": 1,

    "individual_sorted": individual_sorted

}

client_socket.sendall(

    (json.dumps(response) + "\n").encode()

)

# ================================================

# CASE 2

# MERGE BOTH ARRAYS AND SORT

# ================================================

print("\n\n")

print("#" * 70)

print("CASE 2: MERGE BOTH ARRAYS AND SORT")
```

```
print("#" * 70)

# Get both arrays

with lock:

    array1 = client_arrays[1]

    array2 = client_arrays[2]

    merged_array = array1 + array2

print("\nClient 1 Array:", array1)

print("Client 2 Array:", array2)


print("\nMerged Array:")

print(merged_array)

# Make copy

final_array = merged_array.copy()

# Selection Sort

final_sorted = selection_sort(

    final_array,

    "MERGED ARRAY"

)

# Send final sorted array

response = {

    "case": 2,
```

```
        "merged_sorted": final_sorted
    }
    client_socket.sendall(
        (json.dumps(response) + "\n").encode()
    )
    # ------------------------------
    # Close connection
    # ------------------------------

    client_socket.close()
    print("\nClient", client_number, "connection closed.")
# ================================================
# CREATE SERVER SOCKET
# ================================================
server_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)
# ---------------------------------
# Bind server to all network interfaces
# ---------------------------------
```

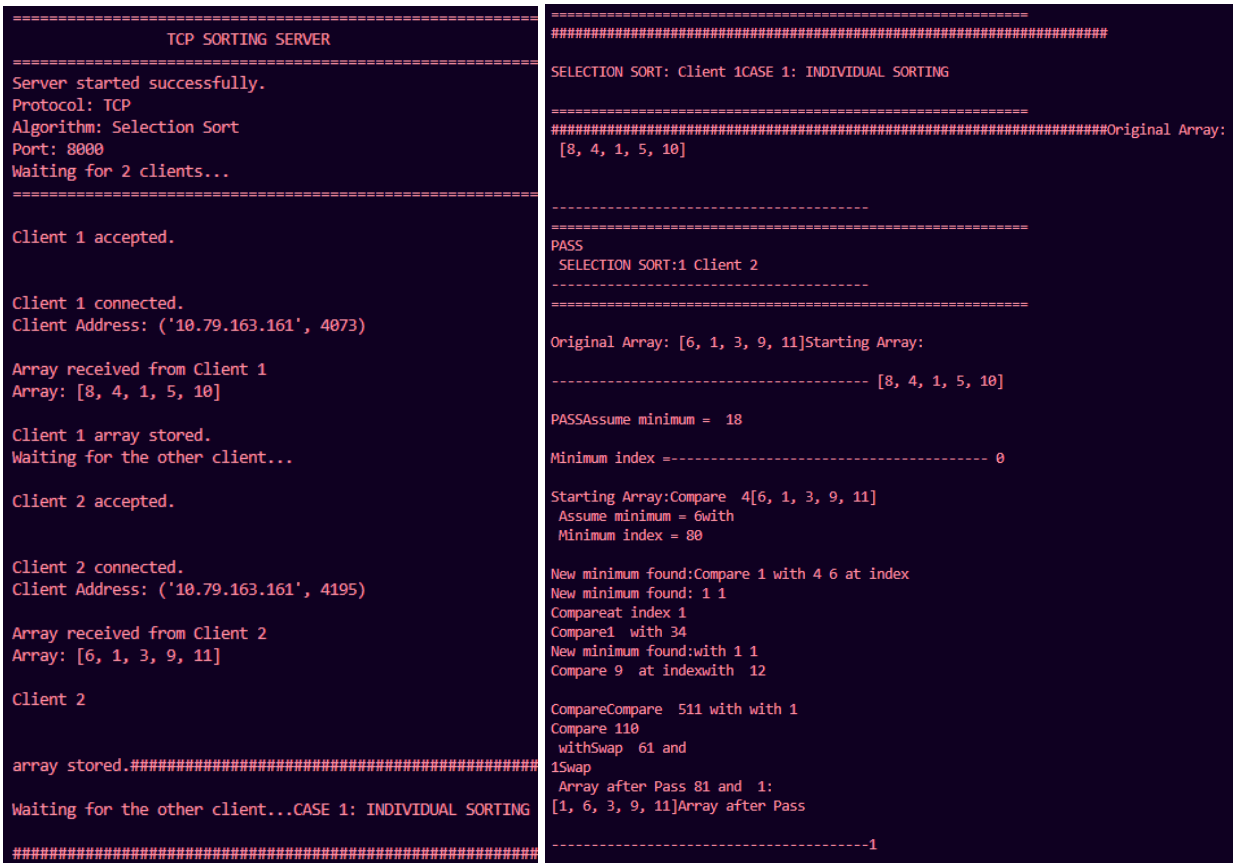
```
server_socket.bind(
    ("0.0.0.0", 8000)
)
# ---------------------------------
# Listen for 2 clients
# ---------------------------------
server_socket.listen(2)
print("\n")
print("=" * 70)
print("                TCP SORTING SERVER")
print("=" * 70)
print("Server started successfully.")
print("Protocol: TCP")
print("Algorithm: Selection Sort")
print("Port: 8000")
print("Waiting for 2 clients...")
print("=" * 70)
# ================================================
# ACCEPT CLIENT CONNECTIONS
# ================================================
client_count = 0
```

```
while client_count < 2:
    client_socket, address = server_socket.accept()
    client_count += 1
    print("\nClient", client_count, "accepted.")
    # Create thread for client
    client_thread = threading.Thread(
        target=handle_client,
        args=(
            client_socket,
            address,
            client_count
        )
    )
    client_thread.start()
```

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

SERVER OUTPUT:



```
============================================================
                TCP SORTING SERVER
============================================================
Server started successfully.
Protocol: TCP
Algorithm: Selection Sort
Port: 8000
Waiting for 2 clients...
============================================================

Client 1 accepted.


Client 1 connected.
Client Address: ('10.79.163.161', 4073)

Array received from Client 1
Array: [8, 4, 1, 5, 10]

Client 1 array stored.
Waiting for the other client...

Client 2 accepted.


Client 2 connected.
Client Address: ('10.79.163.161', 4195)

Array received from Client 2
Array: [6, 1, 3, 9, 11]

Client 2


array stored.##################################################

Waiting for the other client...CASE 1: INDIVIDUAL SORTING

############################################################
```

```
============================================================
##################################################################

SELECTION SORT: Client 1CASE 1: INDIVIDUAL SORTING

============================================================
##################################################################Original Array:
 [8, 4, 1, 5, 10]


----------------------------------------
============================================================
PASS
 SELECTION SORT:1 Client 2
----------------------------------------
============================================================

Original Array: [6, 1, 3, 9, 11]Starting Array:

---------------------------------------- [8, 4, 1, 5, 10]

PASSAssume minimum =  18

Minimum index =---------------------------------------- 0

Starting Array:Compare  4[6, 1, 3, 9, 11]
 Assume minimum = 6with
 Minimum index = 80

New minimum found:Compare 1 with 4 6 at index
New minimum found: 1 1
Compareat index 1
Compare1  with 34
New minimum found:with 1 1
Compare 9  at indexwith  12

CompareCompare  511 with with 1
Compare 110
 withSwap  61 and
1Swap
 Array after Pass 81 and  1:
[1, 6, 3, 9, 11]Array after Pass

----------------------------------------1
```

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Information Technology

```
######################################################################:
CASE 2: MERGE BOTH ARRAYS AND SORT[1, 4, 5, 8, 10]

######################################################################
Final Sorted Array:
[1, 4, 5, 8, 10]
Client 1 Array:
[8, 4, 1, 5, 10]============================================================

Client 2 Array: [6, 1, 3, 9, 11]



Merged Array:

######################################################################[8, 4, 1, 5, 10, 6, 1, 3, 9, 11]

CASE 2: MERGE BOTH ARRAYS AND SORT


============================================================##########################################

SELECTION SORT:
Client 1 Array:  MERGED ARRAY[8, 4, 1, 5, 10]

============================================================Client 2 Array:
Original Array:  [6, 1, 3, 9, 11][8, 4, 1, 5, 10, 6, 1, 3, 9, 11]


Merged Array:
---------------------------------------

[8, 4, 1, 5, 10, 6, 1, 3, 9, 11]PASS 1


---------------------------------------

Starting Array:============================================================
SELECTION SORT:[8, 4, 1, 5, 10, 6, 1, 3, 9, 11]
MERGED ARRAYAssume minimum =
 8============================================================

Original Array: Minimum index =[8, 4, 1, 5, 10, 6, 1, 3, 9, 11]
```

```
Final Sorted Array:Swap  [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]10
and============================================================
 8
Array after Pass 7
Client :  2 connection closed.[1, 1, 3, 4, 5, 6, 8, 10, 9, 11]


---------------------------------------
PASS 8
---------------------------------------
Starting Array: [1, 1, 3, 4, 5, 6, 8, 10, 9, 11]
Assume minimum = 10
Minimum index = 7
Compare 9 with 10
New minimum found: 9 at index 8
Compare 11 with 9
Swap 10 and 9
Array after Pass 8 : [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]

---------------------------------------
PASS 9
---------------------------------------
Starting Array: [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]
Assume minimum = 10
Minimum index = 8
Compare 11 with 10
No swap required.
Array after Pass 9 : [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]

Final Sorted Array: [1, 1, 3, 4, 5, 6, 8, 9, 10, 11]
============================================================

Client 1 connection closed.
PS C:\Users\djsce.student\Desktop\Experiment_8_TCP>
```

Conclusion:

TCP socket communication was successfully implemented between one server and two clients, and arrays were sorted using Selection Sort individually and after merging.

REFERENCES:

1. Data communication and networking by Forouzan , 4th Edition.

2. Computer networks by Tanenbaum